Counting Cheat Sheet

When to use nCr, nPr, 2^n, n(n−1), n!, !n, n(n+1)/2, and n(n−1)/2

Core question: Am I choosing? arranging? taking a subset? counting pairs? counting ranges?

1. The Fast Decision Tree

Ask: Does swapping the selected items create a new answer? If yes, order matters. If no, order does not matter.

Situation	Use	Meaning
Choose exactly r items; order does not matter	`C(n, r)` or `nCr`	Combinations
Choose exactly r items; order matters	`P(n, r)` or `nPr`	Permutations of r items
Each item is either included or not included	`2^n`	All subsets
Pick two different items; order matters	`n(n-1)`	Ordered pairs with $i \neq j$
Arrange all n items	`n!`	All permutations
Arrange all n items; nobody stays in original position	`!n`	Derangements
Count pairs/ranges with $i \le j$	`n(n+1)/2`	Pairs allowing equality; non-empty subarrays
Count pairs with $i < j$	`n(n-1)/2`	Unordered pairs of different items

2. nCr — Choose Without Order

Use when

You choose **r items from n** and the order is irrelevant.

`C(n,r)=n!/(r!(n-r)!)`

Keywords: choose, select, group, team, committee, subset of size r.

Example: Choose 3 problems from 10 → `C(10,3)`.

3. nPr — Choose With Order

Use when

You choose **r items from n** and their positions/ranks matter.

`P(n,r)=n!/(n-r)!`

Keywords: arrange, rank, first/second/third, password without repetition, ordered selection.

Example: Pick gold, silver, bronze from 10 people → `P(10,3)`.

4. 2^n — All Subsets

Use when

Every item has exactly two independent choices: **take it** or **leave it**.

`2^n`

Example: Number of all subsets of an array of size n → `2^n`.

> **Important:** Choose exactly r items → `nCr`. Choose any number of items → `2^n`.

5. n(n−1) — Ordered Pair of Different Items

Use when you pick two different items and order matters.

`n(n-1)=P(n,2)`

Example: Choose a president and vice president from n people → `n(n-1)`.

Pattern: ordered pair `(i,j)` where `i ≠ j`.

6. n! — Arrange Everything

Use when all n items must be arranged.

`n!`

Example: Arrange 5 books on a shelf → `5!`.

If you arrange only r out of n items, use `nPr`, not `n!`.

7. !n — Derangements

Use when you arrange all n items but no item is allowed to stay in its original position.

`!n=(n-1)(!(n-1)+!(n-2))`

Example: Count permutations `p[1..n]` such that `p[i] != i` for every i.

Small values: `!1=0`, `!2=1`, `!3=2`, `!4=9`, `!5=44`.

8. n(n+1)/2 vs n(n−1)/2

`n(n+1)/2`

Use for `i ≤ j`, so equality is allowed.

- Number of non-empty subarrays
- Number of ranges `[l,r]` with `l ≤ r`
- Unordered pairs where choosing the same item is allowed

`n(n-1)/2`

Use for `i < j`, so the two items must be different.

- Number of unordered pairs of different items
- Number of handshakes among n people
- Maximum edges in an undirected graph without self-loops

Memory trick: `<` means `-1`. `≤` means `+1`.

9. Common Competitive Programming Patterns

Problem wording	Formula	Why
Count all subsets of n elements	`2^n`	Each element is in/out
Choose k elements	`C(n,k)`	Order does not matter
Choose k elements and arrange them	`P(n,k)`	Order matters
All permutations of n distinct elements	`n!`	Arrange everything
Pairs (i,j), i != j	`n(n-1)`	Ordered different pair
Pairs i < j	`n(n-1)/2`	Unordered different pair
Pairs i ≤ j or all non-empty subarrays	`n(n+1)/2`	Equality allowed
Permutation with no fixed points	`!n`	Derangement

10. Final Mental Checklist

1. **Fixed number r?** Use `nCr` if order does not matter, `nPr` if order matters.
2. **Any number of items?** Use `2^n`.
3. **Arrange all items?** Use `n!`.
4. **Arrange all with nobody in original place?** Use `!n`.
5. **Two different ordered choices?** Use `n(n−1)`.
6. **Unordered pair with i < j?** Use `n(n−1)/2`.
7. **Range/subarray or i ≤ j?** Use `n(n+1)/2`.